



UNIVERSIDAD
DE SANTIAGO
DE CHILE

Informe Laboratorio 2

Paradigmas de Programación

Paradigma Lógico en Prolog

Nombre: William Bastian Lagos Rivera

RUT: 21.056.415-2

Profesor: Edmundo Leiva

Sección: A-1

Fecha: 19/07/2026

Índice

1. Introducción.....	3
2. Descripción del Problema.....	3
3. Marco Teórico: Paradigma Logico.....	3-4
4. Diseño de la Solución.....	4
5. Aspectos de Implementación.....	5
6. Manual de Usuario y Pruebas.....	5
7. Resultados.....	5
8. Conclusiones.....	6
9. Referencias Bibliográficas.....	7

Introducción

El presente informe explica el proceso de diseño, creación e implementación de un simulador de un juego de cartas coleccionables (TCG) utilizando el lenguaje Prolog. El sistema corresponde a una nueva versión del laboratorio desarrollado anteriormente en Scheme, pero esta vez las reglas del juego fueron expresadas mediante predicados y relaciones propias del paradigma de programación lógico. El documento comienza con una contextualización del problema y del paradigma, continúa con el diseño de los TDAs y finaliza con las instrucciones de uso, las pruebas realizadas, los resultados y una conclusión sobre el trabajo desarrollado.

Problema

El desafío consiste en representar las cartas, los mazos, los jugadores y el estado de una partida de TCG, junto con las reglas necesarias para ejecutar acciones como barajar, robar cartas, jugar Pokémon en la banca, cambiar el Pokémon activo, unir energías, evolucionar y atacar. A diferencia de una solución imperativa, el estado no se modifica directamente: cada predicado recibe un juego de entrada y unifica un nuevo término con el juego resultante. Esto obliga a mantener la consistencia de listas anidadas y a expresar las validaciones como condiciones lógicas.

Marco Teórico: Paradigma Lógico

Programación declarativa: En Prolog se describe qué relaciones deben cumplirse en vez de indicar paso a paso cómo ejecutarlas. Los predicados del laboratorio definen condiciones para que una carta, un mazo o una jugada sean válidos.

Unificación: Permite relacionar términos y obtener valores para variables lógicas. Se utilizó para construir cartas, separar componentes de listas y generar nuevos estados del juego a partir del estado anterior.

Backtracking y búsqueda: Prolog puede intentar distintas cláusulas hasta encontrar una solución que satisfaga las condiciones. Este mecanismo se aprovecha en búsquedas sobre listas, por ejemplo al localizar cartas o verificar la existencia de un Pokémon básico.

Recursión y listas: Las estructuras principales fueron modeladas como listas. La recursión permite recorrerlas, contar copias, repartir cartas, extraer elementos y construir la representación textual del tablero.

Abstracción y modularidad: Los TDAs separan la representación interna de sus operaciones. Los módulos de carta, baraja y juego exportan únicamente los predicados necesarios para coordinar la simulación.

Desarrollo

Se identificaron las entidades críticas del sistema y se organizaron en tres módulos de datos principales, además del archivo central de ejecución:

- **Gestión de cartas:** constructores y selectores comunes para cartas de energía, Pokémon, entrenador y ataques.
- **Gestión de la baraja:** validación de 60 cartas, existencia de Pokémon básicos, límite de copias y barajado determinista.
- **Gestión del juego:** representación de jugadores, zonas del tablero, turno actual y transiciones entre estados.
- **Ejecución y pruebas:** archivo principal que importa los TDAs, define efectos simples y ejecuta una simulación de combate.

Soluciones y TDA

- **TDA Carta y Ataque**

Se utilizó una representación común donde las primeras posiciones de cada carta indican su tipo, expansión, número y nombre. A partir de esta base se agregan los atributos específicos. Las cartas Pokémon incluyen PS, tipo, debilidad, resistencia, costo de retirada, indicador EX, habilidad y lista de ataques. Cada ataque almacena costo, nombre, descripción, daño y un predicado asociado. Los selectores `getTipoCarta/2`, `getExpansion/2`, `getNumero/2` y `getNombre/2` evitan repetir accesos directos a los campos comunes.

- **TDA Baraja**

La baraja corresponde a una lista de 60 cartas. El predicado `createDeck/2` comprueba que el mazo tenga el tamaño exigido, que contenga al menos un Pokémon básico y que ninguna carta distinta de energía aparezca más de cuatro veces. Para el barajado se implementó `shuffleDeck/3`, que extrae recursivamente una carta según la semilla y actualiza dicha semilla mediante `randomPuro/2`. De este modo, una misma semilla genera siempre el mismo resultado, lo que facilita la repetición de pruebas.

- **TDA Juego**

Un Pokémon en juego se representa como `[CartaBase, Energías, Daño, Estado]`. Cada jugador se modela mediante `[ID, Mano, Premios, Mazo, Banca, Activo, Descarte]`, mientras que el juego completo corresponde a `[Jugador1, Jugador2, Turno]`. Esta estructura permite que cada acción reconstruya únicamente las partes que cambian y conserve el resto mediante unificación.

Diseño de la Solución

El flujo principal comienza con `initGame/4`. El predicado baraja ambos mazos, reparte siete cartas a cada participante, vuelve a barajar cuando una mano no contiene un Pokémon básico, separa seis premios y determina el turno inicial a partir de la semilla. Las acciones posteriores reciben el estado actual y producen un nuevo estado sin modificar el término original.

Las operaciones sobre la mano y la banca reutilizan el predicado auxiliar `sacarDeMano/3` para retirar una carta y reconstruir la lista. El cambio de Pokémon activo también valida el costo de retirada y mueve las energías pagadas al descarte. Para los ataques se busca el ataque por nombre, se verifica el número de energías, se calcula el daño considerando debilidad o resistencia y se actualiza el Pokémon defensor. Si queda fuera de combate, se mueve al descarte y el atacante obtiene una carta de premio.

Los efectos de habilidades, ataques y cartas de entrenador se relacionan con predicados que pueden ser ejecutados dinámicamente mediante `call/1` y sus variantes. Este mecanismo permite separar la estructura de la carta de la acción que produce y entrega una base para incorporar efectos más complejos en futuras versiones.

Aspectos de Implementación

El proyecto fue desarrollado para SWI-Prolog versión 8.4 o superior, utilizando predicados estándar del lenguaje. El código se distribuyó en los archivos `tda_carta_21056415_LagosRivera.pl`, `tda_deck_21056415_LagosRivera.pl`, `tda_juego_21056415_LagosRivera.pl` y `main_21056415_LagosRivera.pl`. Los tres archivos de TDA declaran módulos y el archivo principal los incorpora mediante `use_module/1`.

Los predicados incluyen comentarios sobre su finalidad, argumentos y estrategia. La mayor parte del procesamiento se realiza mediante recursión, descomposición de listas, comparación de términos y unificación. No se emplearon bibliotecas externas ni variables globales para guardar el estado de la partida.

Manual de uso y Pruebas

Para ejecutar el laboratorio se deben mantener los cuatro archivos .pl en una misma carpeta y tener SWI-Prolog instalado. Luego se realiza el siguiente procedimiento:

1. Abrir la consola de SWI-Prolog y ubicarse en la carpeta del proyecto.
2. Cargar el archivo principal mediante el predicado consult.

```
?- consult('main_21056415_LagosRivera.pl').
```

3. Cuando la consola responda true, ejecutar el script incorporado.

```
?- test.
```

4. Revisar la secuencia de acciones y la impresión del tablero final.

El script crea tres Pokémon básicos, dos evoluciones, un Pokémon EX, tres cartas de entrenador, una energía y dos ataques. Después configura un combate entre Pikachu y Squirtle, lleva a Mewtwo EX a la banca, une una energía, usa una carta de entrenador, evoluciona a Pikachu en Raichu y ejecuta el ataque Rayo Paralizante. La debilidad de Squirtle duplica el daño y permite comprobar el descarte del Pokémon derrotado y la obtención de un premio.

Resultados

La implementación cubre los constructores principales, la validación y mezcla del mazo, la inicialización de una partida, la impresión del tablero y las acciones centrales de banca, cambio de activo, robo, energía, evolución y ataque. El escenario incluido en main permite seguir una secuencia completa hasta derrotar al Pokémon activo del rival. Al finalizar, Squirtle queda en el descarte, el espacio activo del jugador 2 aparece vacío y el jugador 1 pasa de seis a cinco premios.

Las mecánicas avanzadas quedaron implementadas de forma parcial. Todavía no se registra en el estado si el robo fue la primera acción, si ya se unió una energía o si se utilizó un partidario durante el turno. Las cartas de entrenador y las habilidades ejecutan efectos simples, pero no procesan aún una lista general de argumentos adicionales. También quedaron pendientes algunas condiciones completas de victoria, el tratamiento especial de premios para Pokémon EX y el cambio automático de turno después de atacar. Por esta razón, el sistema funciona como una base modular de gestión de tablero y combate simple, pero no reproduce todavía todas las reglas del juego original.

Conclusión

El paradigma lógico permitió representar las validaciones del TCG como relaciones entre términos, utilizando la unificación y la recursión como mecanismos principales. En comparación con la solución anterior en Scheme, Prolog reduce la necesidad de controlar explícitamente cada paso del algoritmo, ya que una operación tiene éxito cuando se cumplen las condiciones definidas en sus cláusulas. Esto facilita la lectura de reglas como la existencia de un Pokémon básico, el límite de copias o la compatibilidad de una evolución.

La mayor dificultad se presentó al actualizar estructuras anidadas, porque una acción puede afectar al mismo tiempo la mano, la banca, el Pokémon activo, los premios y el descarte. Fue necesario descomponer cada término y construir un nuevo estado manteniendo sin cambios los elementos restantes. A pesar de que algunas reglas avanzadas requieren ampliación, la solución obtenida establece una arquitectura modular, permite ejecutar una simulación de combate y demuestra la aplicación de los conceptos fundamentales del paradigma lógico

10. Referencias Bibliográficas

SWI-Prolog. (s. f.). SWI-Prolog Reference Manual. <https://www.swi-prolog.org/pldoc/man>

SWI-Prolog. (s. f.). Defining a Module. <https://www.swi-prolog.org/pldoc/man?section=defmodule>

SWI-Prolog. (s. f.). call/1. <https://www.swi-prolog.org/pldoc/man?predicate=call%2F1>